

Reviewer Pack — Minimal Geometric Entropy of Affine Hulls and Resolution Pro...

Entry c3c9ed5c42f3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 09:41:50 UTC

Minimal Geometric Entropy of Affine Hulls and Resolution Proof Width

Entry ID: c3c9ed5c42f3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 09:41:50 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

For any given CNF φ with m clauses over n variables, the resolution proof width $w(\varphi)$ is linearly correlated with the minimal geometric entropy $H(\min)(\text{aff}(\varphi))$ of its associated affine hull $\text{aff}(\varphi)$, such that $w(\varphi) = \Theta(H(\min)(\text{aff}(\varphi)))$

Rationale (proposer's reasoning):

The study of geometric entropies has shown promise in quantifying geometric complexity, and an affinity between affine structures and resolution proof processes could reveal novel insights into the complexity of satisfiability problems. This conjecture suggests a potential link between these two fields by proposing that the width of resolution proofs is proportional to the entropy of the associated affine hull.

Taxonomy category: Affine_Geometry_Resolution_Proof_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 82a4436ebb84026a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the computed correlation coefficients (over 30 seeds) are below the threshold of 3, and no seed produces a correlation coefficient above 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Geometric Entropy AND Affine Hulls AND Resolution Proof Width
- Affine Geometry AND Resolution Proof Complexity AND Geometric Entropy - Resolution Proof Width IN AFFINE HULLS AND Geometric Entropy OF Affine Geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = 1 / A[i][i]
            for j in range(n):
                A[i][j] *= factor
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C
```

```

def resolution_width(phi):
    # Simplified DPLL solver to estimate width
    clauses = phi.split('\n')
    variables = set()
    for clause in clauses:
        if clause:
            variables.update(clause.split())
    assignment = {var: random.choice([True, False]) for var in variables}
    width = 0
    for clause in clauses:
        if not clause:
            continue
        unsatisfied_clauses = [c for c in clause.split() if (c[0] == '-' and assignment[c[1:]])]
        if len(unsatisfied_clauses) > width:
            width = len(unsatisfied_clauses)
    return width

def geometric_entropy(phi):
    # Constructive mapping from lines in the projective plane to points in the affine space
    n = phi.count('x')
    m = phi.count('c')
    if n == 0 or m == 0:
        return 0
    entropy = 0
    for i in range(n):
        for j in range(m):
            entropy += math.log2(1 / (n * m))
    return entropy

def generate_cnf(n, m):
    variables = [f'x{i+1}' for i in range(n)]
    clauses = []
    for _ in range(m):
        clause = random.sample(variables + ['-'] + variables, random.randint(2, n))
        clauses.append(' '.join(clause) + ' 0')
    return '\n'.join(clauses)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    for _ in range(5):
        phi = generate_cnf(n, random.randint(1, n))
        width = resolution_width(phi)
        entropy = geometric_entropy(phi)
        if entropy == 0:
            continue
        results.append((width, entropy))

if not results:
    return {
        "metric_name": "resolution_width",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

```

```

    }

    mean_width = sum(w for w, _ in results) / len(results)
    mean_entropy = sum(e for _, e in results) / len(results)
    correlation_coefficient = sum((w - mean_width) * (e - mean_entropy) for w, e in results) / len(results)

    return {
        "metric_name": "resolution_width",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient < 3 and all(c <= 10 for c in results),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len([r for r in results if r["metric_value"] is not None])
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len([r for r in results if r["metric_value"] is not None]))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(r["counterexample"] == "mapping_undefined" for r in results):
        print("RESULT: INCONCLUSIVE mapping_undefined")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4815094.py", line 124, in <module>

result = run_trial(seed)

~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_b4815094.py", line 90, in run\_trial

width = resolution\_width(phi)

~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4815094.py", line 60, in resolution_width

unsatisfied_clauses = [c for c in clause.split() if (c[0] == '-' and assignment[c[1:]]) or (c[0] !=

' and not assignment[c])]

~~~~~

KeyError: 'x1'

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which means it did not complete its intended computation to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 22747        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 17532        |
| 3  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 14022        |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8428         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8744         |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 27577        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8636         |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10660        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16054        |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 11610        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146011 ms total latency. Provider mix: {'ollama\_remote': 10}

*(full prompt+response transcripts available in research/audit/c3c9ed5c42f3.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c3c9ed5c42f3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/c3c9ed5c42f3.tar.gz (if generated)